

Threshold Lab: Comprehensive Architecture & Code Walkthrough

M.Sc. Project (COMP702) — Department of Computer Science, University of Liverpool

Executive Project Overview:

This technical document presents the complete architectural walkthrough, theoretical derivations, and line-by-line implementation details of **Threshold Lab**. Threshold Lab is an interactive Streamlit application developed to calibrate binary classification boundaries under severe class imbalance (0.172% fraud rate) by bridging **Analytical Philosophy (Sorites Paradox & Crispin Wright's 1975 Tolerance Principle)** with **Cost-Sensitive Machine Learning (Elkan 2001)**.

1. Backend Architecture & ML Engines (src/ Directory)

The backend architecture of Threshold Lab is modularized into three core engines within the src/ directory:

Engine Module	File Path	Key Responsibilities
Data Loader	src/data_loader.py	Synthetic ULB generator (0.172% fraud), Kaggle creditcard.csv loader, local auto-detection, PCA feature descriptions.
ML Pipeline	src/ml_engine.py	Logistic Regression & Random Forest pipelines, Platt Scaling (Sigmoid), Isotonic Regression, Brier Score, Log-Loss, PR-AUC.
Sorites Engine	src/sorites_engine.py	Elkan (2001) cost threshold formula $\tau^* = C(FP)/(C(FP)+C(FN))$, continuous cost-curve minimizer, contested zone analysis.

2. Detailed Walkthrough — Tab 1: Sorites Paradox Primer

Theoretical Purpose: Tab 1 introduces the philosophical foundation of the project based on Eubulides of Miletus' *Sorites Paradox* and Crispin Wright's (1975) *Tolerance Principle*. It demonstrates how rigid binary thresholds create paradoxical operational decisions when applied to continuous probability estimates.

Key Features & Components:

- **Interactive Sand Heap Sandbox:** A visual scatter plot simulation where users drag a slider (n = 10 to 2,000 grains). It demonstrates how removing 1 grain fails to flip the predicate 'heap', yet cumulative removals force an abrupt logical transition.
- **Concept Cards:** Explicitly defines **Rigid Decision Threshold (tau)** (crisp binary cutoff) vs **Tolerance Margin (epsilon)** (infinitesimal risk divergence parameter).
- **The Epsilon Dilemma Cards:** Side-by-side transaction cards with assigned numerical values:
 - **Transaction A (TX-89402):** Amount \$842.50, V17 = 14.2 km, V14 = 3.81, Risk Score $p(X_A) = 0.5010$ -> **DECLINED & BLOCKED (FRAUD)**.
 - **Transaction B (TX-89403):** Amount \$841.90, V17 = 14.1 km, V14 = 3.79, Risk Score $p(X_B) = 0.4990$ -> **APPROVED & PASSED (GENUINE)**.
 - Demonstrates that a tiny divergence $\Delta p = 0.0020$ forces opposite business actions.

```
# Code Snippet: Epsilon Dilemma Card Implementation (app.py)
col_card1, col_card2 = st.columns(2)
with col_card1:
    st.markdown(f"""
    <div class="tx-card-a">
    <h4>■ Transaction A (ID: TX-89402)</h4>
```

```

<ul>
  <li><b>Amount:</b> $842.50 | <b>Velocity (V14):</b> 3.81</li>
  <li><b>Calibrated Risk Score p(X_A):</b> {prob_tx_a:.4f}</li>
  <li><b>Decision:</b> <span class="tag-fraud">DECLINED & BLOCKED (FRAUD)</span></li>
</ul>
</div>
""", unsafe_allow_html=True)

```

3. Detailed Walkthrough — Tab 2: Data Explorer & Decision Visualizer

Theoretical Purpose: Tab 2 provides deep empirical data exploration and visualizes how linear models (Logistic Regression) vs non-linear tree ensembles (Random Forest) construct decision surfaces across multi-dimensional feature space.

Key Features & Components:

- **Executive Metric Cards:** Displays Total Transactions (50,000), Genuine (49,914), Fraudulent (86), and Fraud Imbalance Rate (0.172%).
- **Top-Mounted Feature Selector:** Full-width dropdown container placing X-Axis and Y-Axis feature selectors (e.g. V14 vs V17) directly above the 2D plot.
- **2D Meshgrid Decision Surface Plot:** Evaluates raw model probability $p(x)$ across a 45x45 grid.
- **Stratified Fraud Scatter Plotting:** Enforces stratified sampling so ALL fraud points ($y=1$) in the dataset are rendered as prominent **Neon Red Diamonds** (#FF0033, size 11, bold black border). Genuine points are plotted as emerald green dots (#059669).
- **Feature Distribution Overlay & Live Cohen's d:** Plots KDE density histograms for genuine vs fraud and calculates effect size separation (Cohen's d).
- **Model Feature Importance Below:** Stacked linearly below with a sky blue background theme (#F0F9FF).

```

# Code Snippet: Stratified Fraud Points Plotting (app.py)
# Guarantees Fraud Red Diamonds Are ALWAYS Visible Regardless of Imbalance
fr_pts = test_sub_all[test_sub_all['Class'] == 1]
if len(fr_pts) < 15:
    fr_pts = df[df['Class'] == 1]

fig_boundary.add_trace(go.Scatter(
    x=fr_pts[bound_x], y=fr_pts[bound_y], mode='markers',
    name=f'Fraud Transactions (Class 1, n={len(fr_pts)}),
    marker=dict(color='#FF0033', opacity=1.0, size=11, symbol='diamond',
                line=dict(color='#000000', width=1.8)),
    text=hover_text_fr, hoverinfo='text'
))

```

4. Detailed Walkthrough — Tab 3: Elkan's Threshold Lab

Theoretical Purpose: Implements Elkan's (2001) foundational Cost-Sensitive Learning theorem to calculate the global cost-minimizing threshold τ^* .

Mathematical Formulation: Predicting fraud is optimal when Expected Cost of Predict Fraud $\leq$ Expected Cost of Predict Genuine, yielding:

$$\tau^* = C(\text{FP}) / (C(\text{FP}) + C(\text{FN}))$$

Key Features & Components:

- **Economic Terms Briefing Card:** Explains $C(\text{FP})$ (customer friction / SMS fee), $C(\text{FN})$ (chargeback loss), τ^* , and confusion matrix monetary breakdown.
- **Interactive Cost Sliders:** Live adjustments for $C(\text{FP})$ (£1-£500) and $C(\text{FN})$ (£50-£10,000).
- **5 Executive Metrics:** Displays Optimal Threshold τ^* , Total Expected Loss (£), FP Cost, FN Cost, and F1-Score at τ^* .
- **Linear Stacked Visualizations:**
 1. **Dynamic Confusion Matrix Heatmap:** Shows monetary breakdown at τ^* .
 2. **Continuous Cost-Curve Minimizer:** Plots total financial loss across τ in $[0, 1]$ with a golden star marking τ^* .

```
# Code Snippet: Elkan (2001) Threshold Calculation (src/sorites_engine.py)
def compute_elkan_threshold(c_fp, c_fn):
    """Calculates optimal decision threshold tau* = C(FP) / (C(FP) + C(FN))"""
    if (c_fp + c_fn) <= 0:
        return 0.5
    return float(c_fp / (c_fp + c_fn))
```

5. Detailed Walkthrough — Tab 4: Probability Calibration

Theoretical Purpose: Raw machine learning output scores (e.g. uncalibrated SVMs or Random Forest trees) do not reflect true posterior probabilities $P(Y=1|X=x)$. Tab 4 evaluates probability calibration techniques.

Key Calibration Techniques:

- **Platt Scaling (Sigmoid):** Fits a parametric logistic curve: $p(x) = 1 / (1 + \exp(A * s(x) + B))$.
- **Isotonic Regression:** Fits a non-parametric, piecewise monotonic step function.

Key Visualizations & Evaluation:

- **Calibration Briefing Card:** Defines Brier Score Loss, Log-Loss, Platt Scaling, Isotonic Regression, and PR-AUC.
- **Linearly Stacked Graphs:**
 1. **Reliability Diagram (Calibration Curves):** Plots predicted fraud probability vs observed empirical fraud fraction. Diagonal $y=x$ represents perfect calibration.
 2. **Precision-Recall Curves:** Evaluates classification performance under 0.172% fraud imbalance.
- **Quantitative Comparison Table:** Compares Brier Score Loss, Log-Loss, and PR-AUC across Raw, Platt, and Isotonic modes.

```
# Code Snippet: Fitting Calibration Pipelines (src/ml_engine.py)
def fit(self, X_train, y_train):
    self.raw_model.fit(X_train, y_train)
    train_scores = self.raw_model.predict_proba(X_train)[: , 1]

    # Fit Platt Scaling (Sigmoid)
    self.platt_calibrator = CalibratedClassifierCV(self.raw_model, method='sigmoid', cv='prefit')
    self.platt_calibrator.fit(X_train, y_train)

    # Fit Isotonic Regression
    self.isotonic_calibrator = CalibratedClassifierCV(self.raw_model, method='isotonic', cv='prefit')
    self.isotonic_calibrator.fit(X_train, y_train)
```

6. Detailed Walkthrough — Tab 5: The Contested Zone

Theoretical Purpose: Quantifies institutional stakeholder conflict over decision cutoffs and maps the Sorites indeterminacy region.

Key Features & Components:

- **Contested Zone Briefing Card:** Explains Lower Threshold ($\tau_{\min}$) (preferred by Customer Support), Upper Threshold ($\tau_{\max}$) (preferred by Risk Audit), and the Contested Zone $[\tau_{\min}, \tau_{\max}]$.
- **Stakeholder Cost Sliders:** Live adjustments for risk-averse vs friction-averse stakeholder penalties.
- **Interactive Amber-Glowing Scatter Plot:** Visualizes individual transactions trapped within the contested risk interval.
- **CSV Export Feature:** Downloads disputed transactions as `contested_transactions_export.csv`.

```
# Code Snippet: Contested Zone Analysis (src/sorites_engine.py)
def analyze_contested_zone(df, y_probab, c_fp_range, c_fn):
    tau_min = compute_elkan_threshold(c_fp_range[0], c_fn)
    tau_max = compute_elkan_threshold(c_fp_range[1], c_fn)

    in_zone_mask = (y_probab >= tau_min) & (y_probab <= tau_max)
    contested_df = df[in_zone_mask].copy()
    contested_df['Fraud_Probability'] = y_probab[in_zone_mask]

    volume_at_stake = contested_df['Amount'].sum()
    return {
        'tau_min': tau_min, 'tau_max': tau_max,
        'contested_df': contested_df, 'volume_at_stake': volume_at_stake
    }
```

7. UI/UX Customization Summary

Summary of Key Interface Customizations:

- **Top Heading Banner:** Positioned full-width at the very top of the dashboard.
- **Sub-Heading Navigation Tabs:** Rendered directly below the top heading banner.
- **Right-Hand Control Panel:** Positioned on the right side of the screen with a rich Oxford Blue background (#0F172A).
- **Pure White Tooltip Circle Icons:** All question mark help icons in the Control Panel are styled in pure white (FFFFFF).
- **Unified Electric Blue Theme:** Headings and active selected tabs are styled in Electric Blue (#0284C7) with bold dark black text (#000000).
- **Seamless Light & Dark Mode Support:** Full contrast compatibility across all components when toggling Streamlit themes.